A Constructive W-Reflective Number Beyond Contender 5

Conservative semantics, double diagonalization,
and a Lean 4 submission candidate

Research dossier prepared for Vladimir Reshetnikov

October 12, 2026

Abstract

We specify a single constructive natural number N for a Lean adaptation of Cody Roux's biggest-number contest. The construction extends the exact finite-type interpreter behind Contender 5 by recursively generated, well-founded branching-tree types, takes a finite syntactic maximum, and then applies two levels of diagonal growth and forty-two-fold iteration. The mathematical comparison is strict: if $T(0) = 1$ and $T(h+1) = 2^{T(h)}$, then $N > T(C_5) > C_5$. No halting oracle, choice principle, postulated normalization theorem, or evaluation of C_5 enters the definition. A second result supplies an explicit universality threshold: a closed unary program of depth d has its value at n bounded by the syntactic maximum whenever $n \geq \max(32, d+1)$. This yields a precise diagonal separation from the object language. The archive contains a transparent Lean definition and an explicit strict-comparison proof, an independent Python model, executed finite tests, and build/audit scripts. The Lean files have *not* been kernel-checked in the producing runtime, and the contest's 15-second/60-second timing gates remain unverified. Accordingly, this is a mathematically proved construction and a source-level submission candidate, not a certified accepted entry.

Validation boundary. Paper proofs: supplied in full for the central claims. Python: six regression groups passed, including exhaustive enumeration through depth four. Lean: source written without admitted placeholders; compiler execution and an actual empty-axiom report are pending. No measured Lean timing or machine certification is claimed.

Contents

1 The proposed number and the scope of the result **3**

2 Rules and a faithful audit of the reference **3**
2.1 The selector and the numerical maximum 4

3 A finite-type language closed under well-founded trees **4**
3.1 Types and their semantics 4
3.2 Constants and structural elimination 5
3.3 Total casts 5
3.4 Evaluation and conservation 6

4 Exact finite enumeration and its cost **6**
4.1 An explicit conservative guard 7
4.2 Counting without evaluating 8

5 An explicit universal-envelope theorem **8**

6 Comparison proofs without extensionality or evaluation **9**
6.1 The Lean certificate's trust-minimizing design 10

7 A proved margin: a tower of height Contender 5 **10**

8 Transport to other total languages and independence of syntax **11**

9 What was executed, and what remains a validation gate **12**
9.1 Executed tests 12
9.2 Unexecuted kernel and timing checks 12
9.3 Why code shape is not timing evidence 12

10 Contributions, limitations, and further research **13**
10.1 What is contributed 13
10.2 Prioritized next research targets 13

11 Conclusion **14**

A Archive map and reproduction commands **14**

B A compact definition of the outer growth system **15**

1 The proposed number and the scope of the result

The entry is not a Busy Beaver number. It is a closed expression assembled from inductive definitions, structural interpreters, finite lists, and natural-number recursion. Its size is witnessed by a proof, not by attempting to compute its decimal expansion.

Let $M_{\mathsf{T}_W}(n)$ be the maximum natural result of evaluating every raw program of depth at most n in the language T_W defined in Sections 3 and 4; a non-natural result contributes zero. Set

$$B(n) = 1 + \max\{n, M_{\mathsf{T}_W}(n), 2^{n+1}\}. \tag{1}$$

For any $f : \mathbb{N} \to \mathbb{N}$, write $f^{\langle 0\rangle}(x) = x$ and $f^{\langle k+1\rangle}(x) = f^{\langle k\rangle}(f(x))$. Define

$$A_f(0, n) = f(n), \tag{2}$$
$$A_f(k+1, n) = A_f(k, n) + A_f(k, \cdot)^{\langle n+1\rangle}(n+1) + 1, \tag{3}$$
$$J(f)(n) = A_f(n, n), \tag{4}$$
$$R_0(n) = B(n), \qquad R_{k+1}(n) = J(R_k)(n), \tag{5}$$
$$F(n) = R_n(n), \tag{6}$$
$$N = \boxed{F^{\langle 42\rangle}(42)}. \tag{7}$$

The iteration superscript denotes function iteration, not exponentiation. All ranks and iteration counts are ordinary finite naturals, even though their values need not be feasibly calculated.

Theorem 1.1 (Main mathematical comparison)**.** *Let C_5 be the number denoted by* `contender_5` *in the pinned upstream repository, and let $T(0) = 1$, $T(h+1) = 2^{T(h)}$. Then the definition above denotes a natural number, and*

$$N > T(C_5) > C_5.$$

The first part of the proof is the structural totality argument. The strict comparison with C_5 is in Section 6; the tower bound is proved in Section 7. The latter is a deliberately weak lower bound relative to the construction. Its purpose is to certify a substantial margin without appealing to unverified ordinal-growth labels.

No absolute-optimality claim. A "largest number I can devise" is a request for an ambitious entry, not a well-specified optimization theorem. The original rules refer to a reasonable machine, without fixing its input encoding, clock, cache policy, or precise admissible dependency environment. For a fully fixed finite resource model, there may be a finite set of admitted programs and hence a largest denotation. It would be incorrect to argue that successor always preserves a hard timing bound. This dossier does not claim to find that resource-bounded optimum. It provides one explicit, strong, independently motivated construction with exact comparison theorems.

2 Rules and a faithful audit of the reference

The inspected reference is `codyroux/name-the-biggest-number`, commit

```
d56dce013e6264292533efdc5e1de71863c6fb86
```

Its README requires a constructive number, no axioms, a formal comparison, and checking times of at most 15 seconds for the definition and one minute for its comparison proof. It also discourages

simply defining the new entry using the previous named contender [1]. For the proposed adaptation, replace Coq by the pinned native Lean 4.19.0 executable; the timing and validation policy is made explicit in Section 9.

The upstream file attributes Contender 5 to `samuelgruetter`. Its number is not just the value of an ordinary large primitive-recursive expression:

$$C_5 = M_{\mathsf{T}}(42), \qquad M_{\mathsf{T}}(n) = \max\big(\{0\} \cup \{\mathrm{eval}(t) : \mathrm{depth}(t) \le n,\ t \text{ in the old syntax}\}\big). \tag{8}$$

Here the old syntax is the simply typed lambda calculus over naturals with natural recursion at every finite arrow type. There are six constructors: variables, annotated lambdas, application, zero, successor, and an annotated natural recursor. Three details of its executable semantics matter.

First, the variable convention uses *levels from the oldest binding*. The environment is stored with its newest binding at the head, but the lookup index is $|e| - 1 - x$. Thus lookup in $[10, 11, 12]$ returns 12 at index zero, 11 at index one, and 10 at index two. Under newly introduced binders the oldest variable keeps index zero. Replacing this by ordinary newest-first indices changes the number being compared.

Second, the enumeration includes *raw, potentially malformed terms*. The interpreter is total by assigning defaults to bad casts, out-of-range variables, and application of non-functions. Deleting malformed terms and claiming equality with the reference maximum would require an additional proof. We do not make that deletion.

Third, its casts at arrow types are explicit contravariant/covariant wrappers. When both component types agree, an $A \to B$ value is converted to a $C \to D$ value by converting the argument from C to A and the result from B to D. If the component types disagree, the entire cast produces the default value. Our implementation preserves that rule, rather than substituting an arbitrary coercion strategy.

2.1 The selector and the numerical maximum

The upstream implementation uses a running best *term*, initialized at zero, and finally evaluates the selected term. The new implementation uses a numerical maximum fold. This is an implementation difference, not a change in denotation.

Lemma 2.1 (Selector invariant)**.** *Let $s : X \to \mathbb{N}$, let $b \in X$, and let a finite scan start with current score $c = s(b)$. At every step, replace (c, b) by $(s(x), x)$ precisely when $c < s(x)$. After scanning a list L, the selected term has score* $\max(\{c\} \cup \{s(x) : x \in L\})$.

Proof. For an empty list the claim is the invariant $s(b) = c$. For head x and tail L', the update preserves the invariant and changes the current score to $\max(c, s(x))$. Apply the induction hypothesis to L' and associate the finite maximum. Equal-score ties may select different terms, but cannot affect the resulting natural number. In the reference initialization $s(\mathsf{zero}) = 0$. □

Consequently, the scalar port `oldMaximum 42` is exactly the reference number at the mathematical level. The correspondence is a source audit, not a claim that the Coq kernel and Lean kernel have jointly checked a translation.

3 A finite-type language closed under well-founded trees

3.1 Types and their semantics

The type codes are generated by

$$A, B ::= \mathsf{N} \mid A \Rightarrow B \mid \mathsf{W}A.$$

For a set or constructive type X, $\mathsf{W}X$ is the inductively generated type with constructors

$$\mathsf{leaf} : \mathsf{W}X, \qquad \mathsf{node} : (X \to \mathsf{W}X) \to \mathsf{W}X.$$

This is the constant-branching-arity W-tree construction with a leaf alternative. It is not an assertion that all general dependent W-types have been added to our syntax. In particular, $\mathsf{W}\mathbb{N}$ gives naturally indexed, possibly infinitely branching well-founded trees. The syntax also permits such types as $\mathsf{W}(\mathbb{N} \Rightarrow \mathbb{N})$ and $\mathsf{W}((\mathsf{W}\mathbb{N}) \Rightarrow \mathbb{N})$.

The interpretation is

$$[\![\mathsf{N}]\!] = \mathbb{N}, \quad [\![A \Rightarrow B]\!] = ([\![A]\!] \to [\![B]\!]), \quad [\![\mathsf{W}A]\!] = \mathsf{W}[\![A]\!].$$

The Lean type codes themselves inhabit `Type`; their interpretation is a function into `Type`. That type-valued function has the required higher universe in its own type. We do not set `Type : Type`, quantify over a self-containing universe, or identify tree codes with all ordinals. Lean's strict positivity and recursor rules are exactly the mechanisms relevant to these declarations [2, 3].

3.2 Constants and structural elimination

Retain all six old term constructors and add, for every pair of type codes A, R, constants with the following types:

$$\begin{aligned}
&\mathsf{wzero}_A : \mathsf{W}A,\\
&\mathsf{wnode}_A : (A \Rightarrow \mathsf{W}A) \Rightarrow \mathsf{W}A,\\
&\mathsf{wrec}_{A,R} : R \Rightarrow ((A \Rightarrow \mathsf{W}A) \Rightarrow (A \Rightarrow R) \Rightarrow R) \Rightarrow \mathsf{W}A \Rightarrow R.
\end{aligned}$$

The semantics of the last constant is the structural recursor

$$\begin{aligned}
\mathrm{wr}(z, s, \mathsf{leaf}) &= z,\\
\mathrm{wr}(z, s, \mathsf{node}(u)) &= s\big(u,\ a \mapsto \mathrm{wr}(z, s, u(a))\big).
\end{aligned}$$

The step has both the immediate subtrees and their recursively computed values. When A is infinite, the recursive values are supplied as a function; the definition does not eagerly construct an infinite list. This is an inductive recursor, not unrestricted general recursion.

3.3 Total casts

Define default values by zero at natural type, a constant default-returning function at arrow type, and a leaf at tree type. Extend the old cast-pair construction using the following reindexing operation:

$$\begin{aligned}
\mathrm{map}_g(\mathsf{leaf}) &= \mathsf{leaf},\\
\mathrm{map}_g(\mathsf{node}(u)) &= \mathsf{node}(b \mapsto \mathrm{map}_g(u(g(b)))), \qquad g : Y \to X.
\end{aligned}$$

Thus $\mathrm{map}_g : \mathsf{W}X \to \mathsf{W}Y$. The direction is contravariant in the branch index. This distinction is important: covariantly mapping the label type would give the wrong type for the children.

Suppose casts between A and B have components $p : A \to B$ and $q : B \to A$. If the codes A, B pass the structural equality test, the tree cast pair is $(\mathrm{map}_q, \mathrm{map}_p)$. If they fail, both directions return the appropriate default. Mixed top-level shapes also return defaults. At arrow types the earlier wrappers and equality tests are left unchanged.

Lemma 3.1 (Correctness of self-casts)**.** *For every type code A, both components of the cast pair from A to A act extensionally as the identity. Defaults and casts are total.*

Proof. Totality of defaults and cast-pair construction follows by structural recursion on type codes, together with structural recursion defining map. For identity, use induction on A. At natural type the result is immediate. At arrow type, apply the induction hypotheses to the argument and result wrappers and reason pointwise. At tree type, the branch-index conversion is pointwise the identity by the type induction hypothesis. A W-tree induction then shows that the reindexed tree agrees with the original tree: the leaf case is immediate, and at a node the recursively corresponding children agree for every branch index. This argument is an extensional mathematical proof; the main Lean comparison below does not import it or use function extensionality. □

3.4 Evaluation and conservation

Values are dependent pairs (A, v) with $v : [\![A]\!]$. The interpreter recursively traverses a term. A lambda returns a function which interprets the body in the extended environment and casts its result to the annotated return type. An application interprets both operands, casts the argument to the function's input type, and applies it. If the first operand is not an arrow value, the result is natural zero. A closed value at any non-natural type contributes zero to the external score eval.

Theorem 3.2 (Constructive totality and conservative restriction)**.** *All closed expressions used in the construction have constructive semantics. The score of every old raw term under the new interpreter equals its score in the pinned reference interpreter.*

Proof. The type interpretation is structural recursion on a finite code. Casts are as in Lemma 3.1. The term interpretation is structural recursion on its term argument, with its induction result allowed to be a function of the environment. Natural-recursion constants are interpreted by the natural recursor and tree-recursion constants by the W-tree recursor. These are all available total structural operations. Taking a finite maximum of the resulting natural scores is therefore a total operation.

For conservation, inject old type codes by leaving N and arrows unchanged. Under this injection their interpretations coincide. The Boolean equality, default, and two directional cast clauses on these types are the same clauses as the reference. Inject the six old term constructors unchanged. Induct on a raw term, maintaining correspondence of its entire environment. Variables use the same lookup. Zero, successor, and the natural recursor have the same meanings. The lambda case uses the induction hypothesis under an extended environment, and the application case uses it for both operands and then the matching cast. Malformed inputs are covered by the same default clauses; no well-typedness premise has been introduced. The new constants and new non-function tree case never arise in this restriction. Finally, the external score agrees on natural and arrow results. □

This proof does not require a global normalization theorem for Lean or a decision procedure for which arbitrary programs halt. It analyzes the particular structural definitions in the construction.

4 Exact finite enumeration and its cost

Type depths are

$$d(\mathsf{N}) = 1, \qquad d(A \Rightarrow B) = 1 + \max(d(A), d(B)), \qquad d(\mathsf{W}A) = 1 + d(A).$$

The old term depths are retained:

$$\begin{aligned} d(\mathsf{var}(x)) &= x + 2, \\ d(\lambda^{A,B} t) &= 1 + \max\{d(A), d(B), d(t)\}, \\ d(tu) &= 1 + \max(d(t), d(u)), \\ d(\mathsf{zero}) = d(\mathsf{succ}) &= 1, \qquad d(\mathsf{natrec}_R) = 1 + d(R). \end{aligned}$$

For the new constants, use

$$d(\mathsf{wzero}_A) = d(\mathsf{wnode}_A) = 1 + d(A), \qquad d(\mathsf{wrec}_{A,R}) = 1 + \max(d(A), d(R)).$$

These are annotation/syntax depths, not the depths of the inferred full types of constants. The same distinction is already present in the reference natural-recursion constant.

At budget zero both lists are empty. At budget $n+1$, the type list contains N, all arrows whose two component codes belong to the budget-n list, and all W-types of a code in that list. The term list contains variables with $x < n$, lambdas with all annotations and body from budget n, all applications of budget-n terms, the two unannotated constants, and the annotated constants with annotations from budget n. All products are finite list products.

Theorem 4.1 (Exactness and monotonicity)**.** *The generated type and term lists contain exactly the corresponding codes of depth at most their budget. In particular the old lists inject into the new lists at unchanged depth, and*

$$M_{\mathsf{T}}(n) \le M_{\mathsf{T}_W}(n), \qquad M_{\mathsf{T}}(n) \le M_{\mathsf{T}}(n+1), \qquad M_{\mathsf{T}_W}(n) \le M_{\mathsf{T}_W}(n+1).$$

Proof. Induct on the budget. At zero no constructor has permitted depth. At a successor budget, inspection of each constructor's depth formula gives precisely the listed child-budget constraints. The induction hypothesis turns these into membership conditions in the child lists; finite list products and maps supply exactly the desired constructor combinations. The variable constraint is $x + 2 \le n + 1$, equivalently $x < n$. Conversely, every generated constructor meets its depth formula. Old codes remain codes of the new grammar and retain their depths, which proves the conservative inclusion. The maximum statements follow from inclusion of the corresponding finite sets of scores and Theorem 3.2. □

4.1 An explicit conservative guard

The Lean definition uses

$$E(n) = \max(M_{\mathsf{T}}(n), M_{\mathsf{T}_W}(n)), \qquad B(n) = 1 + \max\{n, E(n), 2^{n+1}\}. \tag{9}$$

By Theorem 4.1, $E(n) = M_{\mathsf{T}_W}(n)$, so this is exactly (1). Retaining the redundant maximum makes the main comparison proof short and independent of a mechanization of list inclusion. It does not enlarge the mathematical number. It is important that the independent definition (1) can be stated without the old maximum at all. The source reuses a generic language subsystem, not the closed constant `contender_5`; the latter is declared only in the proof module. This is not a disguised definition $C_5 + 1$.

4.2 Counting without evaluating

Let a_n, b_n count old type and term codes, and u_n, v_n count new ones. Starting with all four zero at $n = 0$, the exact recurrences are

$$a_{n+1} = 1 + a_n^2, \qquad b_{n+1} = n + a_n^2 b_n + b_n^2 + 2 + a_n,$$
$$u_{n+1} = 1 + u_n^2 + u_n, \qquad v_{n+1} = n + u_n^2 v_n + v_n^2 + 2 + 3u_n + u_n^2.$$

Constructor tags and child tuples are unique, so the lists contain no duplicate codes. The final u_n^2 counts the two-annotation tree recursor.

Depth	a_n	b_n	$M_{\mathsf{T}}(n)$	u_n	v_n	$M_{\mathsf{T}_W}(n)$
0	0	0	0	0	0	0
1	1	2	0	1	2	0
2	2	10	1	3	13	1
3	5	146	2	13	308	2
4	26	24 976	3	183	147 129	3

These rows were evaluated by the independent Python model in the archive. The equality of the two maxima at these shallow depths is not evidence of asymptotic equivalence. It is also not presented as evidence for a specific proof-theoretic ordinal. Enumeration at depth 42 was not attempted.

5 An explicit universal-envelope theorem

A useful structural result is available independently of the extra W-type features. It explains why a bounded-depth maximum is much more than a large arithmetic expression.

Let $\ell(0) = 0$ and, for $n > 0$, let $\ell(n)$ be its binary bit length. There is a closed old-language program D with natural input and output whose value is $x \mapsto 2x$, and whose raw depth is 11. In level notation one can take

$$D = \lambda x.\ \mathsf{natrec}_{\mathsf{N}}\ 0\ (\lambda i.\lambda r.\mathsf{succ}(\mathsf{succ}(r)))\ x.$$

In this closed term x has index zero, i index one, and r index two. Its annotations are the evident natural and arrow codes. The depth 11 follows by direct use of the preceding depth formulas. The archived Python model gives the complete raw term and checks its depth and values.

Define closed binary numerals recursively by

$$\overline{0} = \mathsf{zero},$$
$$\overline{n} = D\ \overline{\lfloor n/2 \rfloor} \qquad (n > 0 \text{ even}),$$
$$\overline{n} = \mathsf{succ}(D\ \overline{\lfloor n/2 \rfloor}) \qquad (n > 0 \text{ odd}).$$

Each occurrence of D is evaluated as a closed subterm in the empty environment of a top-level application. Thus the use of absolute variable levels does not silently assume an invalid weakening rule for closed terms embedded under extra binders.

Lemma 5.1 (Depth-efficient numerals)**.** *For every $n \in \mathbb{N}$,* $\mathrm{eval}(\overline{n}) = n$ *and*

$$d(\overline{n}) \le 11 + 2\ell(n).$$

For every $n \ge 32$, moreover, $1 + d(\overline{n}) \le n$.

Proof. The value statement follows by binary recursion, since D doubles its input. For positive n, let $m = \lfloor n/2 \rfloor$; then $\ell(n) = \ell(m) + 1$. The doubling application has depth $1 + \max(11, d(\overline{m}))$. An odd final successor adds one more. The induction hypothesis therefore gives the displayed bound, including the case $m = 0$.

For the last assertion, choose $k \geq 5$ with $2^k \leq n < 2^{k+1}$. The elementary inequality $k + 1 \leq 2^{k-2}$ holds at $k = 5$ and persists by induction. Consequently $2\ell(n) = 2(k + 1) \leq n/2$. Since $n \geq 32$, also $12 \leq n/2$, and hence $1 + d(\overline{n}) \leq 12 + 2\ell(n) \leq n$. □

Theorem 5.2 (Explicit envelope universality)**.** *Let q be a closed raw term whose interpreted value has type $\mathsf{N} \Rightarrow \mathsf{N}$ and denotes $g : \mathbb{N} \to \mathbb{N}$. If q belongs to T_W, then*

$$n \geq \max(32, d(q) + 1) \quad \Longrightarrow \quad g(n) \leq M_{\mathsf{T}_W}(n) < B(n).$$

The corresponding statement with M_{T} holds for an old-language q.

Proof. The closed application $q\,\overline{n}$ evaluates to $g(n)$: the argument cast is natural-to-natural and hence the identity. Its depth is $1 + \max(d(q), d(\overline{n})) \leq n$ by Lemma 5.1 and the stated threshold. The application therefore occurs in the budget-n list. Its score is bounded by that list's maximum. The seed is strictly larger than the maximum by its successor construction. □

Corollary 5.3 (Diagonal separation)**.** *No closed unary term of T_W denotes B. The same is true of F and of the function $Q(n) = F^{\langle n+1 \rangle}(n)$.*

Proof. If a term q denoted B, Theorem 5.2 at any sufficiently large n would yield $B(n) \leq M_{\mathsf{T}_W}(n) < B(n)$. Section 6 proves $F(n) \geq B(n)$ and $F(n) > n$. It follows that $Q(n) \geq F(n) \geq B(n)$. The same contradiction applies to a purported representation of either F or Q. □

This is a separation of *uniform functions*. It is not a claim that the single integer N lacks a numeral in the object language. Every particular natural number has such a numeral; what fails is a single unary program for the uniformly diagonal function.

6 Comparison proofs without extensionality or evaluation

The following elementary preservation argument is deliberately designed not to assume that an arbitrary input function f is monotone.

Lemma 6.1 (Retention and inflation)**.** *For every f, k, n,*

$$A_f(k, n) \geq f(n), \qquad A_f(k + 1, n) > A_f(k, n), \qquad J(f)(n) \geq f(n).$$

Consequently $R_k(n) \geq B(n)$, $F(n) \geq B(n) > n$, and $F(n) > M_{\mathsf{T}}(n)$.

Proof. The row at zero is $f(n)$. The next row adds a natural number and one to the previous row's value at the same input. Induction on k proves the first two claims, and the diagonal instance gives the third. Since $R_{k+1} = J(R_k)$, induction on k proves $R_k(n) \geq R_0(n) = B(n)$. Taking $k = n$ gives the result for F. Finally the seed is a successor of a maximum containing both n and $M_{\mathsf{T}}(n)$. □

The retained summand $A_f(k, n)$ is useful. Omitting it and using only an iterated value at a different input would force extra monotonicity hypotheses for the same-input preservation result. In particular, pointwise increases to a function passed into a higher-order program do not generally increase that program's output. No such false monotonicity principle is used here.

Lemma 6.2 (Iteration with an inflationary function)**.** *If $f(x) \geq x$ for every x, then $f^{\langle m \rangle}(x) \geq x$ for every m, x. For $m \geq 1$, $f^{\langle m \rangle}(x) \geq f(x)$.*

Proof. For the first assertion induct on m. The successor case is $f^{\langle m+1 \rangle}(x) = f^{\langle m \rangle}(f(x)) \geq f(x) \geq x$. The same identity gives the second assertion by applying the first assertion after the initial function call. □

Theorem 6.3 (Strict reference comparison)**.** *The entry in (7) satisfies $C_5 < N$.*

Proof. The complete inequality chain is

$$C_5 = M_{\mathsf{T}}(42) < B(42) \leq F(42) \leq F^{\langle 41 \rangle}(F(42)) = F^{\langle 42 \rangle}(42) = N.$$

The first equality is Lemma 2.1 together with the port audit. The strict inequality and next inequality are Lemma 6.1. The last inequality is Lemma 6.2. No value at depth 42 is expanded. □

6.1 The Lean certificate's trust-minimizing design

The definition module uses a primitive recursively defined `maxN` rather than importing convenient order theorems about maxima. Its left and right inequalities are proved directly by induction. The proof module then uses only elementary natural-order transitivity, successor reasoning, and explicit recursive proof terms. It does not use `simp`, `omega`, `native_decide`, function extensionality, classical choice, or proof placeholders. This design avoids relying on an automation tactic's uninspected axiom footprint. The axioms mechanism and the role of quotient primitives are part of Lean's documented kernel interface [4].

The numerical benchmark is declared *after* the proposed entry, in a separate module:

```
def contender_5 : Nat := oldMaximum 42

theorem contender_5_lt_contender_6 :
    contender_5 < contender_6 :=
  Nat.lt_of_lt_of_le
    (Nat.lt_of_lt_of_le
      (old_lt_seed 42) (seed_le_diagonal 42))
    (le_iterate diagonal self_le_diagonal 41
      (diagonal 42))
```

The last upper bound is definitionally the proposed entry because the forty-two-count iteration unfolds once. Its inputs contain large *unevaluated expressions*; the proof does not contain large computed numerals. This is a source-level explanation of the intended certificate, not an assertion that its elaboration has already been observed to succeed.

7 A proved margin: a tower of height Contender 5

Let $T(0) = 1$ and $T(h+1) = 2^{T(h)}$. Induction gives $T(h) \geq h + 1$, so the tower function is strictly increasing and $T(h) > h$.

Lemma 7.1 (Tower growth from the seed)**.** *For $x \geq 1$ and $m \geq 0$, $B^{\langle m \rangle}(x) \geq T(m)$. Also $B(x) > x$ and $A_B(1, x) > x$.*

Proof. The maximum in the seed contains 2^{x+1}, hence $B(x) > 2^x$ for every natural x. Starting with $x \geq T(0) = 1$, induction on the number of calls gives the tower bound using monotonicity of exponentiation. The remaining assertions follow from the seed's maximum containing x and from $A_B(1, x) > B(x)$. □

Theorem 7.2 (Tower-height lower bound)**.** *For the actual upstream C_5, $N > T(C_5)$.*

Proof. By same-input retention across the rank and row indices,

$$N \geq F(42) = R_{42}(42) \geq R_1(42) = A_B(42, 42) \geq A_B(2, 42). \tag{10}$$

Put $g(x) = A_B(1, x)$ and $t = g(43)$. The reference maximum is nondecreasing, so

$$t > B(43) > M_{\mathsf{T}}(43) \geq M_{\mathsf{T}}(42) = C_5.$$

The definition of row two gives

$$A_B(2, 42) = g(42) + g^{\langle 43 \rangle}(43) + 1 > g^{\langle 43 \rangle}(43) \geq g(g(43)) = g(t).$$

The penultimate inequality holds because g is inflationary and forty-three iterations contain at least two; it does not assume that g is monotone. Finally,

$$g(t) = B(t) + B^{\langle t+1 \rangle}(t + 1) + 1 \geq B^{\langle t+1 \rangle}(t + 1) \geq T(t + 1) > T(C_5).$$

The second inequality is Lemma 7.1, and the last uses $t > C_5$ and strict increase of T. Combine these inequalities with (10). □

This proof uses only the old maximum's elementary depth monotonicity. No unproved comparison with a fast-growing hierarchy indexed by named ordinals is needed. The tower theorem is supplied as a paper proof; the included Lean proof module targets the stricter-than-reference statement, not this additional quantitative theorem.

8 Transport to other total languages and independence of syntax

The construction is reusable, but a fixed depth budget is representation sensitive. A change of variable encoding, recursor annotations, or primitive constants can alter a particular bounded maximum. The following result states what a compiler actually certifies.

Proposition 8.1 (Compiler transport)**.** *Suppose two total scored languages have finite depth enumerators and a translation τ which preserves natural scores and satisfies $d(\tau(t)) \leq a\, d(t) + b$, where $a, b \in \mathbb{N}$. Then*

$$M_{\mathcal{L}}(n) \leq M_{\mathcal{K}}(an + b).$$

Proof. Every budget-n source term translates to a budget-$(an + b)$ target term with the same score. Take maxima over the finite source list. The default zero is also below the target maximum. □

Our old-to-new inclusion has $a = 1, b = 0$. An optimization which prunes malformed terms would not automatically have this property; it needs an explicit score-preserving translation or a proof that every removed score is dominated by a retained one. A program which merely shares an informal name with a System-T evaluator is insufficient evidence of such a comparison.

There is also a useful warning about interpretation. A total interpreter for a strictly weaker object language is admissible. An alleged total self-interpreter for all expressions in a language containing its own diagonal construction would contradict Corollary 5.3. The W-language does not contain a primitive denoting its own bounded maximum. The maximum and diagonal operators are defined in the Lean metalanguage. This separation is the precise reason the construction does not become a semantic Berry paradox.

9 What was executed, and what remains a validation gate

9.1 Executed tests

The archive's independent Python interpreter follows the cast-pair rules and level-based environments, and generates both lists of raw terms. Six regression groups passed in the producing runtime. They cover: malformed applications and lookup defaults; exhaustive list generation, uniqueness, inclusion, and maximum scores through depth four; closed binary numerals from zero through 256 and the depth-11 doubling term; tree recursion and identity reindexing, including a function-indexed example; two exact formulas for the first rows over successor; and resource-guard behavior.

For the row tests, $s(n) = n + 1$ gives the identities

$$A_s(1, n) = 3n + 4, \qquad A_s(2, n) = 3^{n+1}(n + 3) + 3n + 3.$$

Indeed $x \mapsto 3x + 4$ iterated k times yields $3^k(x + 2) - 2$. Substitution in (3) proves the second formula. These formulas were checked for $n = 0, \ldots, 6$. The JSON evidence records the actual interpreter version, counts, test outcome, and elapsed time. That Python elapsed time is *not* a Lean checking time.

The test harness has explicit operation and integer-size guards. These guards belong only to the finite test implementation; they do not supply a hidden fuel parameter to the mathematical evaluator or the Lean definition. Attempting to enumerate depth 42 with the harness is rejected rather than falsely reported as completed.

9.2 Unexecuted kernel and timing checks

There was no Lean executable installed in the producing runtime. Attempts to retrieve a usable toolchain did not establish an executable local installation. The evidence file therefore records `kernel_checked: false`, `timing_verified: false`, and `status: not_run`. This limitation is an engineering/verification gap, not an assumed mathematical conjecture. It prevents a claim that the archive is already a compliant contest submission.

The intended native toolchain is `leanprover/lean4:v4.19.0`; the release is pinned for reproducibility, not represented as the latest Lean release [5]. With that toolchain installed, the check script separately measures:

1. definition-module checking, including loading prebuilt `Init` and writing its `.olean`, against 15 seconds;
2. comparison-module checking, importing the checked definition module, against 60 seconds;
3. a separate `#print axioms` audit of both the entry and its theorem.

The script fails closed on a missing or different compiler, a compile error, an unrecognized axiom-report format, or an exceeded gate. It never executes `#eval contender_6`. Elan's toolchain-file mechanism and Lean's standard installation instructions are documented by the project [6, 7].

9.3 Why code shape is not timing evidence

It is plausible that a short structural definition and symbolic inequality certificate will type-check quickly, but plausibility is not a benchmark. Elaboration heuristics, structural recursion recognition, definitional equality, module loading, and library changes can affect the result. The source has been organized to make these checks small and auditable; a successful timed native run is still required. The 15-second limit applies to checking a definition, not to reducing that definition to a decimal numeral.

10 Contributions, limitations, and further research

10.1 What is contributed

The specific construction combines a faithful raw-term reference port, closure under simple W-tree types at arbitrary finite nesting, an explicit bounded-depth universality threshold, and a retained-value double diagonal whose comparison proof needs no function equality. The independent characterization (1) and the conservative guard (9) explain exactly how to obtain both a previous-number-free mathematical definition and a short comparison certificate.

Finite maxima of total interpreters and diagonal growth are established ideas, and the repository already implements the crucial first step. This dossier claims the explicit construction, proofs, and integration artifacts developed here; it does not claim historical priority for diagonalization, W-types, or System-T-style semantics. Nor does it claim that its object language has a particular named proof-theoretic ordinal. Such a label would require an additional analysis of this exact recursor and cast system.

10.2 Prioritized next research targets

1. Complete the native kernel audit. Run the pinned project, resolve any elaboration issues, and record a native 15-second/60-second result and actual empty-axiom outputs. This is the immediate remaining condition for treating the source as a submitted formal entry. It should not be replaced by additional numerical experiments.

2. Mechanize conservative inclusion and the tower bound. The paper proves both statements. A purely syntactic inclusion proof can avoid function extensionality entirely because old and new terms share the same data constructors. It would justify deleting the conservative guard from the source while keeping a short comparison theorem. The tower theorem can likewise be proved using natural-order induction and monotonicity of exponentiation.

3. Internalize the old envelope in the W-language. Construct a closed T_W unary term computing M_{T}, or prove a more appropriate separation statement. Such an internalization would give a genuine uniform reflection theorem from the old language into the new one. The mere addition of W-type syntax does not establish that internalization, and it is not assumed in the present comparison. A promising route is a concrete evaluator with a constructively generated tree of justified recursive calls.

4. Optimize the numeral threshold. The depth-11 doubling program gives the explicit threshold 32. Smaller closed arithmetic combinators, a more careful binary construction, or sharing-aware syntax may improve the threshold. Any claimed improvement must preserve the reference's level convention and annotation depth, rather than changing the cost model silently.

5. Analyze the arithmetical strength of nested branching types. Compare the natural functions definable using $\mathsf{W}\mathbb{N}$, then $\mathsf{W}(\mathbb{N} \Rightarrow \mathbb{N})$, then arbitrary finite tree/type nesting. This requires actual translations or separation arguments; a suggestive ordinal name alone would not settle the question. The resulting compiler depth bounds can be fed directly into Proposition 8.1.

6. Compress enumeration without weakening the guarantee. Sharing type codes and subterms, streaming finite products, and carrying a numeric maximum instead of a best-term witness are natural implementation improvements. More aggressive pruning requires a domination proof for removed scores, especially for malformed terms whose default conversions can still produce nonzero natural values.

7. Escalate reflection with explicit totality certificates. A further object language could contain verified evaluators for earlier languages, followed by another bounded maximum. Every added evaluator needs its own totality argument; unrestricted self-interpretation is blocked by the diagonal theorem. The relevant research problem is to build a transparent hierarchy of such certified interpretations while keeping the checking cost inside a fixed budget.

11 Conclusion

The proposed entry is the specific natural number $N = F^{\langle 42 \rangle}(42)$ from (7). Its mathematical definition does not invoke the closed value of Contender 5, and its conservative implementation keeps the old syntax only as a redundant component of a strictly richer language's finite envelope. The proof gives the explicit lower bound $N > T(C_5)$, while the universality result identifies a concrete eventual domination threshold for every unary program of the object language. The delivered files distinguish these proved mathematical results and executed finite tests from the still-unperformed Lean kernel and timing validation. That distinction is essential to the contest's formal and computational meaning.

A Archive map and reproduction commands

The important paths are:

Path	Purpose
`article/main.tex`, `article/main.pdf`	This article and its compiled PDF.
`lean/BigNumber/Definitions.lean`	Total language, finite maxima, and entry.
`lean/BigNumber/Proofs.lean`	Explicit proof of strict comparison with the reference port.
`lean/BigNumber/Audit.lean`	Entry and theorem axiom audit commands.
`lean/BigNumber/Smoke.lean`	Optional shallow evaluations, separate from timing gates.
`python/reference_model.py`	Independent executable semantics.
`python/test_reference.py`	Six executed regression groups.
`scripts/check_lean.py`	Pinned native compilation and checking-time gates.
`evidence/`	Actual Python and source-audit evidence; honest Lean not-run status.
`upstream/`	Source provenance and upstream MIT license.

From the archive root:

```
python3 python/test_reference.py --json evidence/python_tests.json
python3 scripts/source_audit.py
python3 scripts/check_lean.py
bash scripts/build_pdf.sh
```

The third command requires the pinned Lean executable. The final command uses `pdflatex` twice. Neither invokes the proposed number's evaluation.

B A compact definition of the outer growth system

The interpreter and enumeration are supplied in the full source. Once the seed has been defined, the actual growth mechanism is short:

```
def iterate {A : Type} (f : A -> A) : Nat -> A -> A
  | 0, x => x
  | k+1, x => iterate f k (f x)

def rows (f : Nat -> Nat) : Nat -> Nat -> Nat
  | 0, n => f n
  | k+1, n =>
      rows f k n + iterate (rows f k) (n+1) (n+1) + 1

def jump (f : Nat -> Nat) (n : Nat) : Nat := rows f n n

def ranks : Nat -> Nat -> Nat
  | 0 => seed
  | k+1 => jump (ranks k)

def diagonal (n : Nat) : Nat := ranks n n

def contender_6 : Nat := iterate diagonal 42 42
```

No mathematical exponentiation in this listing is evaluated while merely stating the definition. The arrow syntax here is printed in ASCII for easy copying; the archived Lean source uses Lean's equivalent Unicode notation.

References

[1] Cody Roux and contributors. *Name the biggest number*. Repository README and `Contender.v`; Contender 5 attributed in the source to `samuelgruetter`. Pinned commit `d56dce013e6264292533efdc5e1de71863c6fb86`. https://github.com/codyroux/name-the-biggest-number. Inspected October 12, 2026.

[2] Lean project. *Lean Language Reference: Inductive Types*. https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/. Consulted October 12, 2026; the executable target is separately pinned.

[3] Lean project. *Lean Language Reference: Recursive Definitions*. https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/. Consulted October 12, 2026.

[4] Lean project. *Init.Prelude API*, including the documented quotient axiom and primitive interfaces. https://lean-lang.org/doc/api/Init/Prelude.html. The proof source avoids depending on quotient soundness or extensionality.

[5] Lean project. *Lean 4.19.0 release*, May 1, 2025. https://github.com/leanprover/lean4/releases/tag/v4.19.0. This is the designated validation target, not an asserted local installation.

[6] Lean project. *Elan: the Lean version manager*. https://github.com/leanprover/elan. Toolchain pinning documentation.

[7] Lean project. *Manual installation*. https://lean-lang.org/install/manual/. Consulted October 12, 2026.